Enterprise AI Knowledge Assistant

Production Architecture & Development Roadmap

Goal

Build an enterprise-grade Agentic AI Knowledge Assistant using only free and open-source technologies to demonstrate authentication, RAG, LangChain, LangGraph, MCP, BullMQ, pgvector, Ollama, and asynchronous AI workflows.

Technology Stack

Frontend

- React
- Vite
- Tailwind CSS

Backend

- Express.js
- PostgreSQL
- Redis
- BullMQ

AI Service

- Python
- FastAPI
- LangChain
- LangGraph
- Ollama
- MCP Client

AI Models

- Qwen3 or Llama3 via Ollama

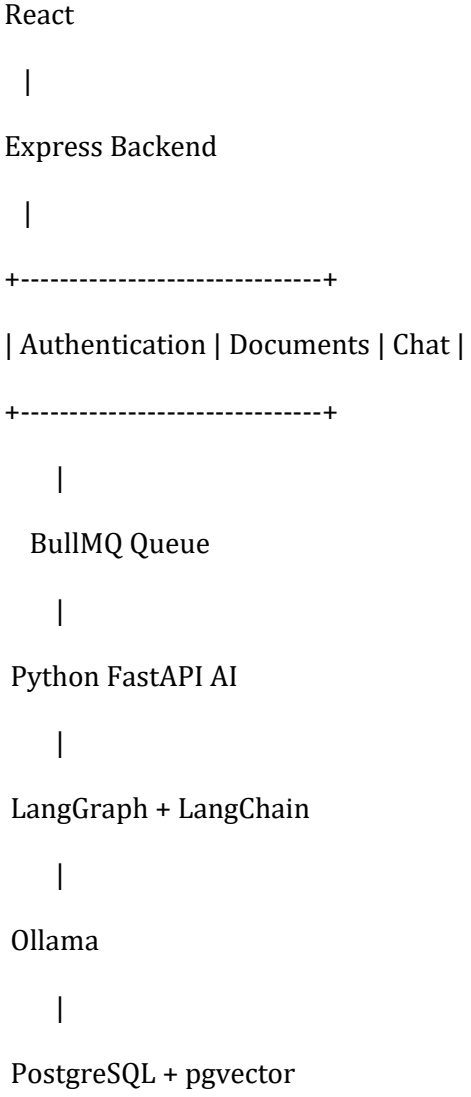
- nomic-embed-text embeddings via Ollama

Storage

- Local filesystem (uploads/)

- PostgreSQL + pgvector

High-Level Architecture



Backend Folder Structure

backend/

src/

auth/

users/

documents/

chat/

queues/

workers/

middleware/

config/

utils/

uploads/

tests/

Python AI Folder Structure

python-ai/

app/

agents/

graphs/

rag/

tools/

memory/

services/

api/

tests/

PostgreSQL Tables

users

- id

- username

- email
- password
- created_at

documents

- id
- user_id
- filename
- filepath
- status
- size
- uploaded_at

document_chunks

- id
- document_id
- chunk_index
- chunk_text
- embedding (pgvector)
- metadata

chat_sessions

- id
- user_id
- title
- created_at

chat_messages

- id

- session_id

- role

- content

- created_at

processing_jobs

- id

- document_id

- status

- started_at

- completed_at

- error

Development Roadmap

Phase 1

- Authentication (Completed)

  - Register

  - Login

  - Refresh Token

  - Logout

  - JWT

Phase 2

- Document Upload

- Upload PDF/TXT/DOCX
- Store locally
- Save metadata in PostgreSQL
- Create BullMQ job

Phase 3

- BullMQ Worker
  - Consume upload job
  - Call Python AI service

Phase 4

- Document Processing
  - Read document
  - Extract text
  - Clean text
  - Chunk text
  - Generate embeddings
  - Store vectors in pgvector
  - Mark document READY

Phase 5

- LangChain RAG
  - Load documents
  - Split text
  - Generate embeddings
  - Vector search

Phase 6

- Chat API
  - User question
  - Vector search
  - Build prompt
  - Ollama response

Phase 7

- Conversation Memory
  - Store chat history in PostgreSQL

Phase 8

- LangGraph
  - Router
  - Retriever
  - Memory
  - LLM
  - Response

Phase 9

- MCP Tools
  - Search documents
  - List documents
  - Document details
  - Delete document

Phase 10

- Supervisor Agent
  - Retriever Agent
  - Research Agent
  - Summarizer Agent

Phase 11

- Reflection Agent

Phase 12

- Planning Agent

Phase 13

- Streaming Responses

Phase 14

- Hybrid Retrieval (BM25 + Vector Search)

Phase 15

- Observability
  - Prompt logs
  - Retrieved chunks
  - Responses
  - Latency
  - Errors

Final Agent Workflow

User Question

|

Authentication

|

Load Conversation

|

LangGraph Supervisor

| | |

Memory Retriever MCP Tools

|

Build Prompt

|

Ollama

|

Reflection Agent

|

Final Answer

|

Store Conversation

Interview Highlights

- JWT authentication with Express and PostgreSQL

- Local document ingestion pipeline

- BullMQ background processing

- LangChain RAG implementation

- pgvector semantic search

- LangGraph agent orchestration

- MCP tool integration

- Local LLM inference with Ollama

- End-to-end enterprise AI architecture using free/open-source technologies